JAVA 2

Module PKG

Packages et visibilité

Au programme

1. Packages et dossiers
2. La visibilité de package
3. Champ `public` ou accesseur
4. Réduire la surface publique
5. Découper en packages
6. Quatrième niveau : `protected`

1. Un package, un dossier



Ranger une classe

La ligne `package` ouvre toujours le fichier

Chaque segment du nom est un dossier sur le disque

Du général au précis : organisation, projet, module

```
// src/main/java/be/ephec/biblio/modele/Livre.java

package be.ephec.biblio.modele;

public class Livre {
    // ...
}
```

Le nom complet d'une classe

Le nom complet d'une classe commence par son package

L'instruction `import` permet d'écrire le nom court

Les classes de `java.lang` se passent d'`import`

```
package be.ephec.biblio.service;

import be.ephec.biblio.modele.Livre;   // nom complet
import java.time.LocalDate;

public class Emprunt {
    private Livre livre;                // nom court
    private LocalDate dateRetour;
}
```

Un sous-package est un package à part

Les noms de packages forment une arborescence, mais pas les droits

Un package n'a aucun accès privilégié à ses sous-packages, ni l'inverse

Deux sous-packages voisins sont deux packages distincts, et étrangers

```
be.ephec.biblio                Main
be.ephec.biblio.modele         Livre, Auteur
be.ephec.biblio.service        Emprunt, Catalogue

// trois packages, sans lien de parente
```

Deux dossiers, un seul package

Maven range le code dans `src/main/java` et les tests dans `src/test/java`

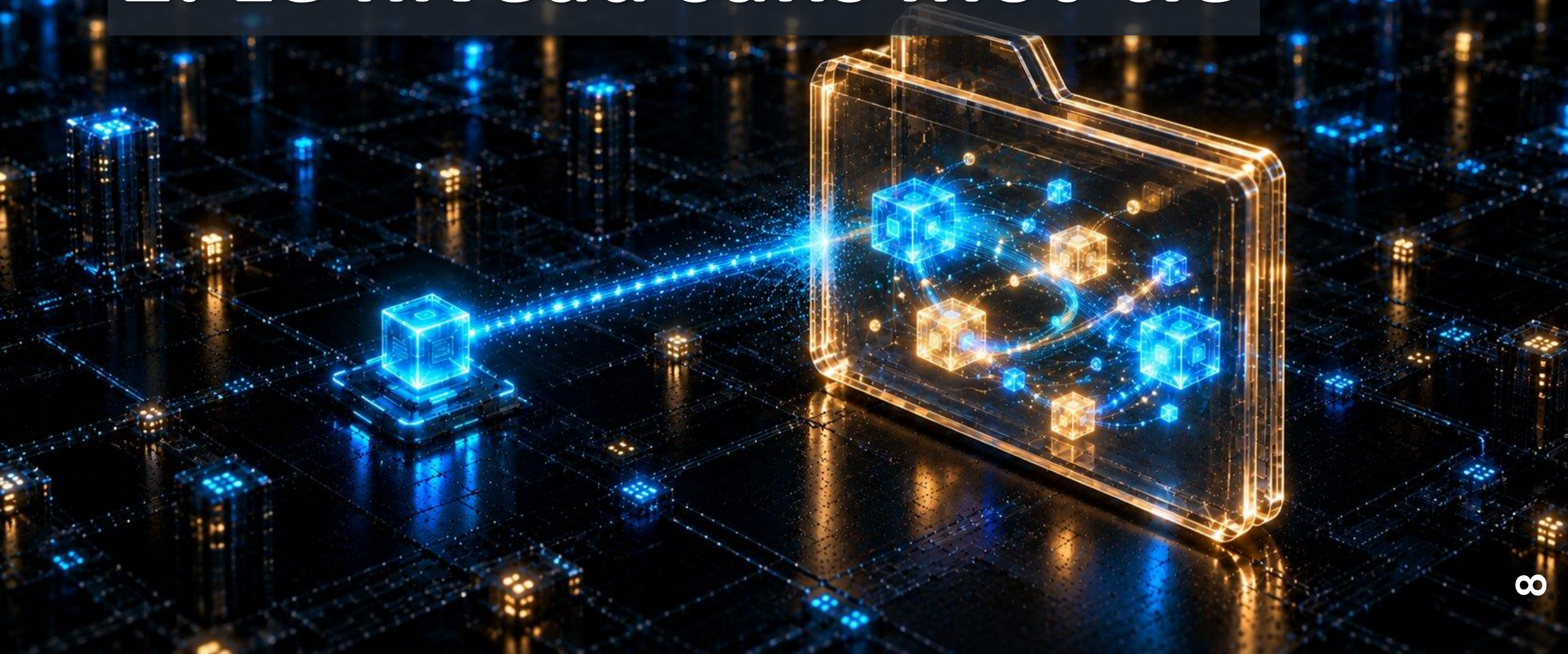
Une classe de test déclare le même package que la classe qu'elle teste

Pour le compilateur, elles sont voisines : deux dossiers, un seul package

```
src/main/java/be/ephec/biblio/modele/Livre.java
src/test/java/be/ephec/biblio/modele/LivreTest.java

// les deux fichiers commencent par la meme ligne :
package be.ephec.biblio.modele;
```

2. Le niveau sans mot-clé



Qui peut atteindre le membre d'une classe

Un membre d'une classe, c'est l'un de ses champs ou l'une de ses méthodes

Le niveau `private` : le code de la classe seul

Le niveau `public` : tout le programme

Sans modificateur : le package, et nulle part ailleurs

```java
package be.ephec.biblio.modele;

public class Livre {

    private String isbn;              // la classe Livre
    String cote() { ... }             // le package modele
    public String titre() { ... }     // tout le programme
}
```

Le niveau qu'on choisit sans le savoir

Sans mot-clé : on l'obtient en n'écrivant rien

Il s'appelle visibilité de package, ou par défaut

Il vaut pour les membres, les constructeurs, la classe

```
package be.ephec.biblio.modele;

class Fiche {                    // invisible hors du package
    String cote;                 // idem
    Fiche(String cote) {         // idem
        this.cote = cote;
    }
}
```

Démonstration



Démonstration

Déménager une classe, et voir ce qui casse

Déroulé : Demo_PKG

Exercices

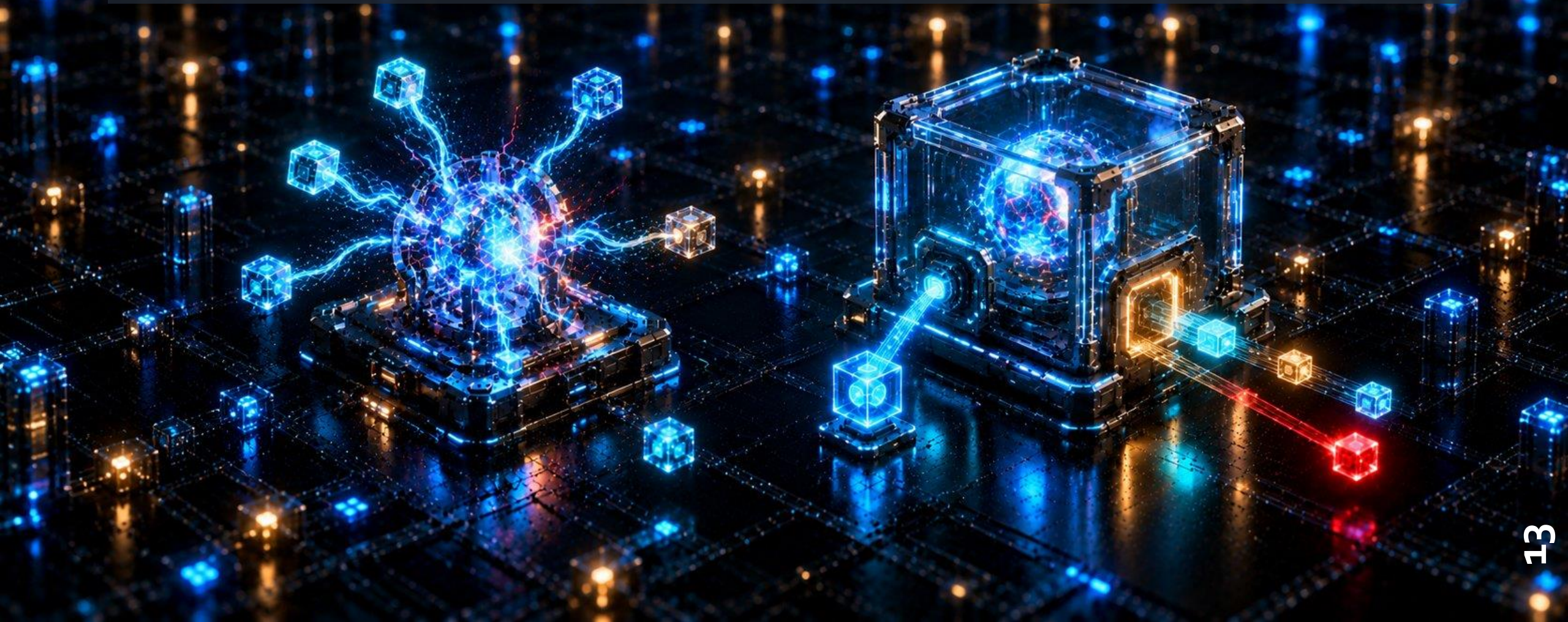


PKG.1 Ce que voit une classe du même package

fascicule PKG

Expérimenter d'abord : le test ne dit pas quelle ligne est fausse

3. Champ public ou accesseur



Ce qu'un champ `public` promet

Un champ `public` se lit et s'écrit, par n'importe qui

La classe ne sait plus qui le modifie, ni comment

Son nom et son type ne peuvent plus changer

```
public class Livre {

    public int exemplaires;
}

// ailleurs, sans que Livre le sache
livre.exemplaires = -3;
```

Ce que l'accesseur garde

Un accesseur `public` ne donne que la lecture

La classe peut calculer, formater, refuser la valeur

Son corps change sans qu'aucun appelant ne le sache

```
public class Livre {

    private int exemplaires;
    private int empruntes;

    public int getDisponibles() {       // hier : un champ
        return exemplaires - empruntes;
    }
}
```

Exercices



PKG.2 Ce que voit une classe d'un autre package

fascicule PKG

Concevoir : CC-OBJ.2

4. La surface publique



Chaque membre `public` est une promesse

Ce qui est `public` peut être utilisé n'importe où, donc ne peut plus changer librement

Fermer un membre ouvert casse le code des autres ; ouvrir un membre fermé ne casse rien

D'où la règle : partir du plus fermé, et n'ouvrir que sur un besoin réel

Choisir, membre par membre

Le niveau `private` d'abord : le cas le plus fréquent

La visibilité de package si une voisine en a besoin

Le niveau `public` pour ce que les autres appellent

```
public class Catalogue {

    private Livre[] livres = new Livre[100];

    public void ajouter(Livre livre) { ... }
    public Livre chercher(String isbn) { ... }

    boolean estCoherent() { ... }       // pour les tests
    private void agrandir() { ... }
}
```

Une seule classe publique par fichier

Une classe est publique ou de package, rien d'autre

Un fichier contient au plus une classe publique, et porte son nom

C'est une règle du compilateur, pas une convention

```
// fichier Catalogue.java

public class Catalogue { ... }

class Rayon { ... }              // permis : visible dans le package

public class Etagere { ... }     // erreur de compilation
```

Exercices



PKG.3 Réduire la surface publique d'une classe

fascicule PKG

Concevoir : CC-CLS.2

5. Découper en packages



Ranger par responsabilité

Un package regroupe des classes qui changent pour les mêmes raisons

Un découpage courant compte trois couches : le modèle, les services, l'application

Le modèle contient les données du domaine, et ne connaît personne

```
be.ephec.biblio.modele          Livre, Auteur
be.ephec.biblio.service         Emprunt, Catalogue
be.ephec.biblio.application     Main
```

Les imports dessinent les dépendances

Les `import` d'une classe disent de quoi elle dépend

Dans un programme bien découpé, les dépendances vont toutes dans le même sens

Une dépendance qui remonte signale une responsabilité rangée dans la mauvaise couche

```
application  ->  service  ->  modele

// les classes du modele n'importent
// ni les services, ni l'application
```

Déplacer une classe avec l'outil

Le refactoring `Move`, touche `F6`, déplace le fichier et corrige la ligne `package`

Il ajoute les `import` dans toutes les classes qui l'utilisent

Un fichier déplacé à la main garde une ligne `package` fausse, et une copie en laisse deux

Exercices



PKG.4 Découper un programme en packages

fascicule PKG

Concevoir : CC-CLS.1

6. Le quatrième niveau



Le rappel du module ABS

Le niveau `protected` ouvre un membre aux sous-classes

Il l'ouvre aussi au package, ce qu'on oublie souvent

Son effet propre : une sous-classe d'un autre package

```
package be.ephec.biblio.modele;
public abstract class Document {
    private final String cote;
    protected int pages;

    protected Document(String cote) {
        this.cote = cote;
    }
}
```

Les quatre niveaux

Niveau	Classe	Package	Sous-classe ailleurs	Reste
`private`	oui	non	non	non
(rien)	oui	oui	non	non
`protected`	oui	oui	oui	non
`public`	oui	oui	oui	oui

Choisir `protected`

Un membre `protected` engage envers toutes les filles

Un champ `protected` pose la question du champ `public`

Il convient aux méthodes que les filles écrivent

```
public abstract class Document {

    private int pages;                      // la mere

    protected int getPages() {              // les filles
        return pages;
    }

    protected abstract String format();  // idem
}
```

Exercices



PKG.5 protected, ou ce que l'héritage ouvre

fascicule PKG

L'exercice ABS.4 vous laissait une décision de visibilité : vous pouvez la trancher